

Modélisation du Web et algorithme PageRank

Rémi Parrot

27 avril 2026

Table des matières

1	Introduction	2
2	Modélisation du Web	2
2.1	Représentation du Web par un graphe orienté	2
2.2	Matrice d'adjacence	2
2.3	Matrice de transition	2
2.4	Limites du modèle	3
3	La matrice de Google	3
3.1	Principe de la téléportation	3
3.2	Définition de la matrice de Google	3
3.3	Propriétés de la matrice G	4
3.4	Interprétation probabiliste	4
4	Modélisation par une chaîne de Markov	4
4.1	Vecteur de probabilité	4
4.2	Évolution du système	4
4.3	Recherche d'un état stationnaire	4
4.4	Interprétation	5
5	Existence et unicité du PageRank	5
5.1	Propriétés spectrales	5
5.2	Théorème de Perron-Frobenius	5
5.3	Conséquence pour le PageRank	5
6	Implémentation en Python	6
6.1	Structure du programme	6
6.2	Génération du graphe	6
6.3	Construction de la matrice de transition	6
6.4	Construction de la matrice de Google	7
6.5	Calcul du PageRank	7
7	Expériences numériques	8
7.1	Génération d'un graphe aléatoire	8
7.2	Convergence de l'algorithme	8
7.3	Visualisation des scores PageRank	9
8	Optimisations	10
8.1	Utilisation de matrices creuses	10
8.2	Influence du paramètre α	10
9	Conclusion	12

1 Introduction

Avec l'essor du Web, les moteurs de recherche sont devenus des outils incontournables pour naviguer dans une masse d'information qui ne cesse de croître. Le problème, c'est que le nombre de pages indexées est tel qu'il faut bien trouver un moyen de les ordonner. Pas seulement par pertinence thématique, mais aussi par importance relative. C'est précisément ce que propose l'algorithme *PageRank*, développé par Sergey Brin et Larry Page à la fin des années 1990 pour le moteur de recherche Google.

Le principe de base est assez intuitif : une page Web est d'autant plus importante qu'elle est citée par d'autres pages, et surtout par des pages elles-mêmes bien référencées. Cette logique circulaire conduit naturellement à modéliser le Web comme un graphe orienté, où chaque page correspond à un sommet et chaque lien hypertexte à une arête orientée.

À partir de ce modèle, on peut voir la navigation d'un utilisateur comme celle d'un *surfeur aléatoire* qui se déplace de page en page en cliquant sur des liens au hasard. Son comportement suit alors une dynamique probabiliste que l'on formalise à l'aide d'une chaîne de Markov. Le classement des pages revient alors à trouver la distribution stationnaire de cette chaîne, c'est-à-dire la fréquence limite avec laquelle chaque page est visitée.

Ce projet a pour but de détailler cette modélisation et l'algorithme qui en découle. On commencera par construire le modèle de graphe orienté et la matrice de transition associée au surfeur aléatoire. On introduira ensuite la matrice de Google, qui corrige certaines limitations du modèle de base, notamment les puits et les composantes déconnectées, et garantit l'existence d'une distribution stationnaire unique. Pour finir, on implémentera le calcul du PageRank en Python et on observera son comportement sur des graphes générés aléatoirement.

2 Modélisation du Web

2.1 Représentation du Web par un graphe orienté

On modélise l'ensemble des pages du Web par un graphe orienté $G = (V, E)$.

- Chaque sommet représente une page web.
- Chaque arête orientée représente un lien hypertexte.

Si une page j contient un lien vers une page i , on note $j \rightarrow i$.

Cette modélisation permet d'utiliser des outils d'algèbre linéaire et de théorie des graphes pour étudier la structure du Web.

2.2 Matrice d'adjacence

Soit n le nombre de pages.

On définit la matrice d'adjacence $A \in \{0, 1\}^{n \times n}$ par :

$$a_{ij} = \begin{cases} 1 & \text{si la page } j \text{ pointe vers la page } i, \\ 0 & \text{sinon.} \end{cases}$$

Ainsi, la colonne j de la matrice A représente l'ensemble des liens sortants de la page j .

2.3 Matrice de transition

On suppose maintenant qu'un internaute parcourt le Web de manière aléatoire : à chaque étape, il choisit uniformément un des liens sortants de la page courante.

On définit alors la matrice de transition P par :

$$P_{ij} = \begin{cases} \frac{1}{\deg(j)} & \text{si } j \rightarrow i, \\ 0 & \text{sinon,} \end{cases}$$

où $\deg(j)$ désigne le degré sortant de la page j , c'est-à-dire :

$$\deg(j) = \sum_{i=1}^n a_{ij}.$$

La matrice P vérifie la propriété suivante :

$$\sum_{i=1}^n P_{ij} = 1$$

pour toute colonne j telle que $\deg(j) \neq 0$.

Ainsi, P est une matrice stochastique par colonnes.

2.4 Limites du modèle

Ce modèle présente cependant deux difficultés principales :

- Certaines pages peuvent ne posséder aucun lien sortant.
- Le graphe peut être composé de plusieurs composantes non connectées, empêchant l'existence d'une distribution stationnaire unique.

Ces problèmes seront résolus dans la section suivante grâce à l'introduction de la matrice de Google.

3 La matrice de Google

3.1 Principe de la téléportation

Le modèle précédent, basé uniquement sur la matrice de transition P , peut poser des problèmes d'existence et d'unicité de la distribution stationnaire. Pour y remédier, on introduit un mécanisme dit de téléportation. À chaque étape, le surfeur a deux comportements possibles :

- suit un lien sortant avec probabilité α ,
- se téléporte vers une page choisie uniformément au hasard avec probabilité $1 - \alpha$.

Le paramètre α vérifie :

$$0 < \alpha < 1.$$

En pratique, on choisit souvent une valeur proche de 0,8.

3.2 Définition de la matrice de Google

On définit la matrice de Google G par :

$$G = \alpha P + \frac{1 - \alpha}{n} \mathbf{1},$$

où :

- P est la matrice de transition,
- n est le nombre de pages,
- $\mathbf{1}$ désigne la matrice dont toutes les entrées valent 1.

Ce qui revient, entrée par entrée, à écrire :

$$G_{ij} = \alpha P_{ij} + \frac{1 - \alpha}{n}.$$

3.3 Propriétés de la matrice G

La matrice G possède deux propriétés importantes :

- Toutes ses entrées sont strictement positives.
- La somme des coefficients de chaque colonne vaut 1.

G est donc une matrice stochastique stricte. C'est cette propriété qui garantit l'existence et l'unicité d'une distribution stationnaire, comme on le verra dans la section suivante.

3.4 Interprétation probabiliste

La matrice G décrit un processus de Markov modélisant la navigation d'un internaute aléatoire. À chaque étape :

- avec probabilité α , il suit un lien hypertexte,
- avec probabilité $1 - \alpha$, il choisit une page uniformément au hasard.

Ce second comportement, la téléportation, assure que le surfeur peut atteindre n'importe quelle page du graphe, ce qui élimine les problèmes de réductibilité rencontrés avec P seule.

4 Modélisation par une chaîne de Markov

Pour modéliser la navigation d'un internaute sur le Web, on adopte une interprétation probabiliste du graphe en le décrivant comme une chaîne de Markov.

4.1 Vecteur de probabilité

On introduit le vecteur :

$$x_k = (x_k^{(1)}, \dots, x_k^{(n)}) \in \mathbb{R}^n$$

où $x_k^{(i)}$ représente la probabilité que le surfeur se trouve sur la page i à l'instant k . Par définition, x_k est un vecteur de probabilité et vérifie :

$$x_k^{(i)} \geq 0 \quad \text{et} \quad \sum_{i=1}^n x_k^{(i)} = 1.$$

4.2 Évolution du système

La transition d'un instant k à l'instant $k+1$ est gouvernée par la matrice de Google G , selon la relation :

$$x_{k+1} = Gx_k.$$

Cette écriture traduit le fait que la position du surfeur à l'instant $k+1$ ne dépend que de sa position à l'instant k , c'est précisément la propriété de Markov.

4.3 Recherche d'un état stationnaire

On cherche un vecteur x^* vérifiant :

$$x^* = Gx^*.$$

Un tel vecteur est appelé *distribution stationnaire* de la chaîne de Markov. Il correspond à un régime d'équilibre où la distribution de probabilité ne se modifie plus d'une étape à l'autre.

4.4 Interprétation

La composante $x^{*(i)}$ a une signification concrète : elle représente la proportion du temps que le surfeur passe sur la page i lorsque le nombre d'étapes tend vers l'infini. Autrement dit, x^* capture la fréquence limite de visite de chaque page. Les pages associées aux plus grandes valeurs de $x^{*(i)}$ sont naturellement considérées comme les plus importantes. C'est sur cette idée que repose le classement PageRank.

5 Existence et unicité du PageRank

On cherche un vecteur de probabilité $x \in \mathbb{R}^n$ tel que :

$$Gx = x.$$

Un tel vecteur est appelé vecteur propre associé à la valeur propre 1. Il représente la distribution stationnaire du processus de Markov défini par la matrice G .

5.1 Propriétés spectrales

La matrice G vérifie les propriétés suivantes :

- G est une matrice stochastique par colonnes.
- Toutes ses entrées sont strictement positives.

Ces deux conditions réunies permettent d'invoquer le théorème de Perron-Frobenius, ce qui est précisément l'intérêt de la construction de G .

5.2 Théorème de Perron-Frobenius

Le théorème de Perron-Frobenius affirme que si une matrice carrée est strictement positive, alors :

- elle admet une valeur propre dominante réelle et positive,
- cette valeur propre est égale à 1 dans le cas d'une matrice stochastique,
- le vecteur propre associé est unique à un facteur multiplicatif près,
- ce vecteur peut être choisi à composantes strictement positives.

5.3 Conséquence pour le PageRank

On est donc assuré de l'existence d'un unique vecteur x^* vérifiant :

$$Gx^* = x^*$$

avec :

$$x_i^* \geq 0 \quad \text{et} \quad \sum_{i=1}^n x_i^* = 1.$$

Ce vecteur correspond à la distribution stationnaire du surfeur aléatoire. Autrement dit, à la fréquence limite avec laquelle chaque page est visitée. Ses composantes constituent les scores de PageRank.

Ainsi, le problème du classement des pages revient à calculer ce vecteur propre associé à la valeur propre 1.

6 Implémentation en Python

Pour illustrer concrètement le modèle présenté, on a implémenté l'algorithme PageRank en Python. L'implémentation s'appuie principalement sur `numpy` pour les calculs matriciels et `networkx` pour la manipulation et la visualisation des graphes.

Dans un premier temps, on travaille avec des matrices pleines, ce qui simplifie l'implémentation et permet de mieux suivre chaque étape de l'algorithme. La question des matrices creuses, plus réalistes au regard de la structure réelle du Web, sera abordée dans un second temps.

6.1 Structure du programme

Le programme est organisé en plusieurs fonctions Python correspondant aux différentes étapes du calcul du PageRank :

- `graph_alea(n)` : génère une matrice d'adjacence aléatoire.
- `calcul_P(A,n)` : construit la matrice de transition.
- `calcul_G(A,n)` : construit la matrice de Google.
- `surfer_alea(n)` : génère un vecteur de probabilité initial.
- `pagerank(G,n)` : calcule le vecteur PageRank par itération.

Ce découpage permet de suivre clairement les différentes étapes du calcul et de séparer l'analyse de la complexité de chaque partie.

6.2 Génération du graphe

Le Web est modélisé par une matrice d'adjacence $A \in \{0,1\}^{n \times n}$, générée aléatoirement dans notre programme pour simuler un réseau de pages web. Chaque coefficient a_{ij} vaut 1 si la page j possède un lien vers la page i , et 0 sinon. Cette matrice sert de point de départ pour construire le graphe orienté associé et calculer les degrés sortants, nécessaires à l'étape suivante.

La génération de la matrice nécessite de remplir n^2 coefficients, ce qui donne une complexité en $O(n^2)$.

6.3 Construction de la matrice de transition

À partir de A , on construit la matrice de transition P définie par :

$$P_{ij} = \begin{cases} \frac{1}{\deg(j)} & \text{si } j \rightarrow i \\ 0 & \text{sinon} \end{cases}$$

où $\deg(j)$ désigne le degré sortant de la page j . En pratique, ce calcul revient à parcourir les colonnes de A et à normaliser chacune par la somme de ses entrées non nulles :

```
def calcul_P(A,n):  
  
    P = np.zeros((n,n))  
  
    for j in range(n):  
  
        deg = np.sum(A[:,j])  
  
        if deg != 0:  
            for i in range(n):  
                if A[i][j] == 1:  
                    P[i][j] = 1/deg  
  
    return P
```

Complexité La fonction parcourt l'ensemble des colonnes de la matrice et examine leurs coefficients. Deux boucles de taille n , le coût total est donc en $O(n^2)$.

6.4 Construction de la matrice de Google

La matrice de Google est ensuite obtenue à partir de P via la formule :

$$G = \alpha P + \frac{1 - \alpha}{n} \mathbf{1}.$$

Le paramètre α est fixé à 0,8 dans notre implémentation : Il règle l'équilibre entre navigation par liens et téléportation aléatoire.

On l'implémente de cette manière :

```
def calcul_G(A,n):  
  
    P = calcul_P(A,n)  
    U = np.ones((n,n))  
  
    G = alpha*P + (1-alpha)/n * U  
  
    return G
```

Complexité La construction de la matrice nécessite, la construction de P en $O(n^2)$, celle de U en $O(n^2)$ aussi, ainsi que des opérations sur les n^2 coefficients de la matrice. Ce qui donne une complexité en $O(n^2)$ également.

6.5 Calcul du PageRank

On construit le vecteur initial x_0 en créant un tableau de taille n , rempli d'entier aléatoires entre 0 et 1, que l'on normalise ensuite avec la somme des élément du tableau pour obtenir un vecteur de probabilité. La complexité de `surfer_alea(n)` est donc en $O(n)$.

Le vecteur PageRank est ensuite calculé par la méthode d'itération suivante :

$$x_{k+1} = Gx_k.$$

On répète le produit matrice-vecteur jusqu'à convergence. Le critère d'arrêt retenu, pour ε choisi arbitrairement, est :

$$\max_i |x_{k+1,i} - x_{k,i}| < \varepsilon.$$

Cette méthode est simple à mettre en œuvre et converge relativement vite en pratique.

L'implémentation Python est la suivante :

```
def pagerank(G,n):  
  
    x = surfer_alea(n)  
  
    xnext = np.dot(G,x)  
    diff = np.max(np.abs(x - xnext))  
  
    while diff > epsilon:  
  
        x = xnext  
        xnext = np.dot(G,x)  
  
        diff = np.max(np.abs(x - xnext))  
  
    return x
```

Complexité Chaque itération correspond à un produit matrice vecteur, dont le coût est en $O(n^2)$, et la boucle est itérée k fois, k étant le nombre d'itérations nécessaires pour atteindre la convergence. La complexité de l'algorithme est donc en $O(kn^2)$.

7 Expériences numériques

Afin d'étudier le comportement de l'algorithme PageRank, nous avons réalisé plusieurs expériences numériques sur des graphes générés aléatoirement.

Dans un premier temps, nous utilisons des matrices pleines afin de simplifier l'implémentation. L'utilisation de matrices creuses, plus adaptées à la structure réelle du Web, sera discutée dans la section suivante.

7.1 Génération d'un graphe aléatoire

Nous générons une matrice d'adjacence aléatoire $A \in \{0, 1\}^{n \times n}$.

Dans les expériences suivantes, pour mieux visualiser nous choisissons arbitrairement :

$$n = 10 ; \alpha = 0,8 ; \varepsilon = 10^{-16}$$

Le graphe orienté correspondant peut être visualisé grâce à la bibliothèque `networkx`.

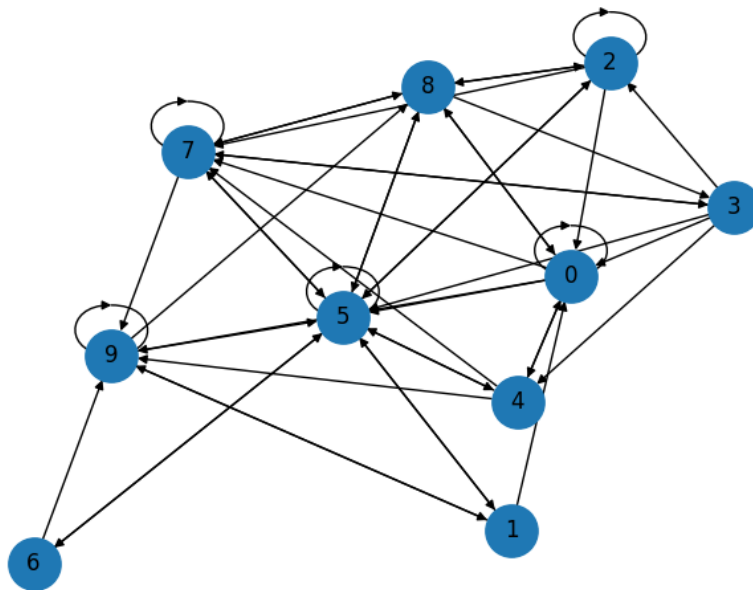


FIGURE 1 – Exemple de graphe orienté généré aléatoirement

7.2 Convergence de l'algorithme

L'algorithme PageRank repose sur l'itération

$$x_{k+1} = Gx_k.$$

Nous observons que la suite (x_k) converge rapidement vers la distribution stationnaire. La figure suivante représente l'évolution de l'erreur entre deux itérations successives.

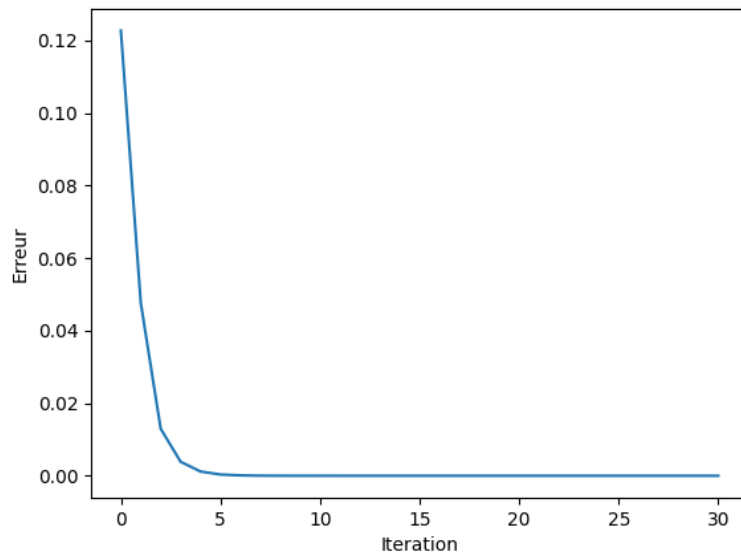


FIGURE 2 – Convergence de l’algorithme PageRank

7.3 Visualisation des scores PageRank

Une fois le vecteur PageRank calculé, il est possible de visualiser l’importance relative des pages en représentant la taille des sommets proportionnellement à leur score.

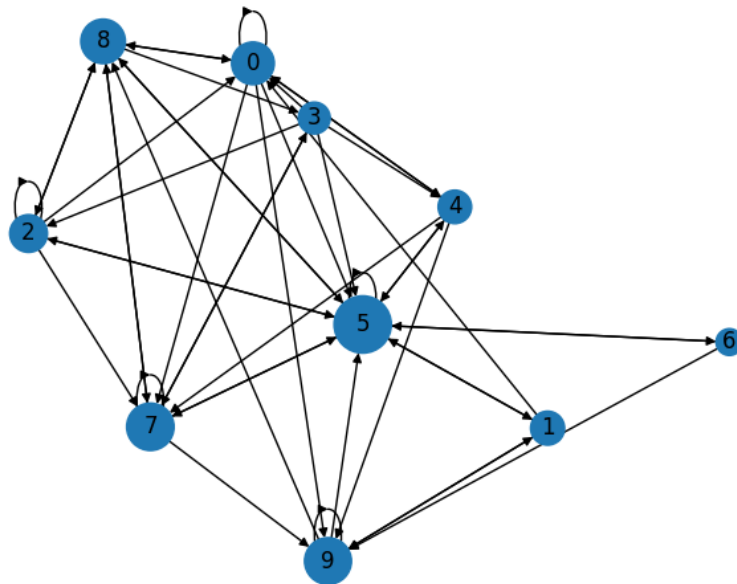


FIGURE 3 – Visualisation du PageRank sur un graphe

On observe que certaines pages apparaissent plus importantes que d’autres, ce qui correspond aux sommets recevant davantage de liens.

8 Optimisations

8.1 Utilisation de matrices creuses

Dans l'implémentation précédente, les matrices sont stockées sous forme pleine de taille $n \times n$, ce qui représente un coût mémoire de l'ordre de $O(n^2)$. Pour de grandes valeurs de n , cela devient vite problématique. D'autant plus que, dans un réseau web réel, chaque page ne pointe que vers un nombre limité d'autres pages. La matrice d'adjacence est donc très majoritairement composée de zéros, ce qui rend la représentation pleine inefficace et non réaliste.

En passant à une représentation creuse, seules les entrées non nulles sont stockées. Le produit matrice-vecteur de l'itération PageRank ne dépend alors plus de n^2 , mais uniquement du nombre d'arêtes m du graphe. La complexité par itération tombe à $O(m)$, et dans notre cas on a bien :

$$m \ll n^2$$

pour de très grandes valeurs de n , ce qui permet de réduire considérablement le temps de calcul.

Pour quantifier cet écart, on a mesuré le temps moyen d'exécution de l'algorithme pour les deux représentations. Pour chaque taille $n = 200, 400, \dots, 2000$, nous avons généré des graphes aléatoires et exécuté l'algorithme 2000 fois pour chaque représentation (dense et creuse). Le temps moyen obtenu est représenté sur la figure suivante :

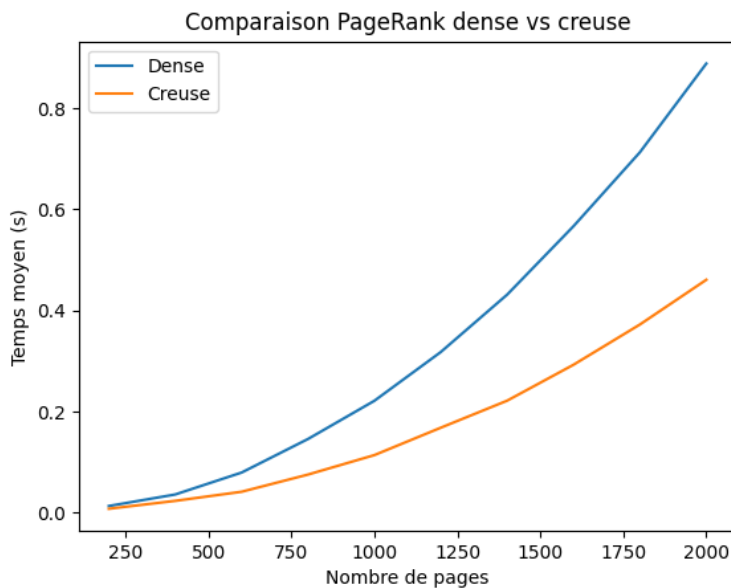


FIGURE 4 – Comparaison du temps de calcul entre matrices denses et creuses

8.2 Influence du paramètre α

Le paramètre α contrôle la probabilité que le surfeur suive un lien hypertexte plutôt que de se téléporter vers une page aléatoire. Il joue un rôle sur deux aspects distincts : le comportement du modèle et la vitesse de convergence de l'algorithme.

La figure suivante montre l'évolution de l'erreur $\|x_{k+1} - x_k\|_\infty$ au fil des itérations, pour différentes valeurs de α et $n = 200$.

Sans grande surprise, les valeurs faibles de α convergent plus vite : la téléportation vers une page aléatoire domine, ce qui homogénéise rapidement la distribution. À l'inverse, des valeurs proches de 1 accordent plus de poids à la structure du graphe, au prix d'une convergence plus

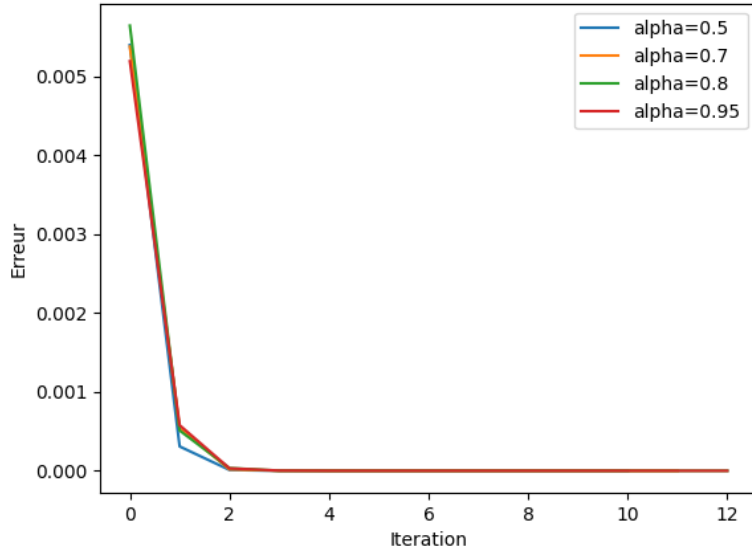


FIGURE 5 – Influence du paramètre α sur la convergence

lente. En pratique, $\alpha = 0,8$ constitue un compromis raisonnable, et c'est la valeur couramment retenue.

On s'intéresse maintenant à l'effet de α sur la distribution des scores PageRank elle-même. Pour cela, on fixe un graphe (plein), de petite taille ($n = 10$) et on calcule le vecteur x^* pour plusieurs valeurs de α .

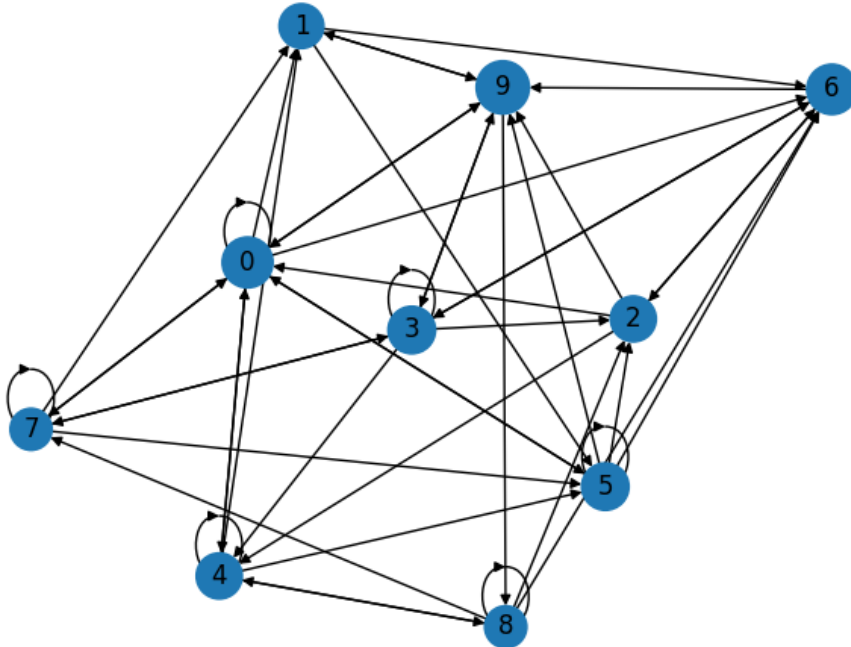


FIGURE 6 – Distribution du PageRank pour $\alpha = 0,6$

Pour $\alpha = 0,6$, les scores sont relativement proches les uns des autres, ce qui reflète un comportement assez uniforme. La téléportation efface en partie les différences de structure. Pour

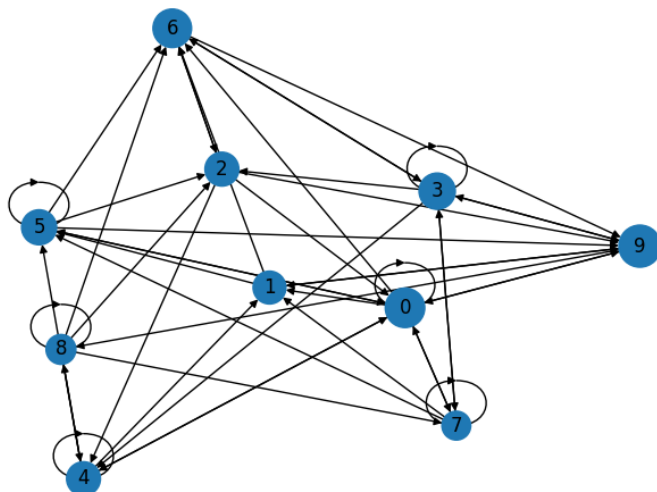


FIGURE 7 – Distribution du PageRank pour $\alpha = 0,9$

$\alpha = 0,9$ en revanche, certaines pages se détachent nettement : la topologie du graphe reprend le dessus, et les sommets bien connectés voient leur score augmenter de façon marquée. Le paramètre α agit donc bien comme un curseur entre exploration aléatoire et fidélité à la structure des liens.

9 Conclusion

Le classement de pages web à grande échelle pourrait sembler relever avant tout de la puissance de calcul brute. Face à un graphe de plusieurs milliards de nœuds, deux stratégies s’offrent en effet : augmenter les ressources disponibles, ou trouver une modélisation qui rende le problème structurellement plus simple. PageRank emprunte résolument la seconde voie.

Plutôt que d’analyser localement chaque lien, l’algorithme exploite une représentation probabiliste du graphe via les chaînes de Markov. Le classement émerge alors d’une propriété globale. La distribution stationnaire est capturée par une équation propre, aussi compacte qu’efficace. Ce changement de point de vue est, rétrospectivement, ce qui rend l’approche aussi élégante que puissante.

Cela dit, la pertinence mathématique ne suffit pas. Comme on l’a vu avec la comparaison entre les matrices denses et les matrices creuses, le passage à une représentation adaptée à la structure réelle du graphe réduit drastiquement le coût de chaque itération, de $O(n^2)$ à $O(m)$, ce qui change tout dès que n dépasse quelques milliers de pages.

Le paramètre α illustre de son côté un compromis inhérent au modèle : donner trop de poids à la téléportation (α faible) efface les différences de structure ; trop peu (α proche de 1) ralentit la convergence. La valeur $\alpha = 0,8$ couramment retenue en pratique n’est pas complètement arbitraire. elle reflète un équilibre empiriquement validé.

Le modèle a toutefois ses limites. Il repose sur une vision assez schématique du comportement des utilisateurs, sans tenir compte du contenu des pages, des requêtes formulées, ni des préférences individuelles. PageRank constitue une base mathématique solide, mais les moteurs de recherche modernes l’ont depuis longtemps enrichi de nombreuses autres couches de traitement.

Ce projet aura au moins montré une chose : des outils relativement accessibles, tels que les graphes orientés, les matrices stochastiques ou encore l’itération de puissance, suffisent à traiter un problème d’une complexité réelle, à condition de les combiner avec soin.